BÀI 3. TÌM HIỂU CÁC KIỂU DỮ LIỆU CÓ TRONG C++

1. Kiểu số nguyên

Bảng sau liệt kê các kiểu dữ liệu số nguyên phổ biến trong ngôn ngữ lập trình C++

Kiểu dữ liệu	Kích thước (byte)	Giá trị có thể lưu	Phạm vi giá trị có thể lưu
short	2	Số nguyên	-32,768 -> 32,767
unsigned short	2	Số nguyên không âm	0 -> 65,535
int	4	Số nguyên	-2,147,483,648 -> 2,147,483,647
unsigned int	4	Số nguyên không âm	0 -> 4,294,967,295
long long	8	Số nguyên	-9223372036854775808 -> 9223372036854775807
unsigned long long	8	Số nguyên không âm	0 -> 18446744073709551615

Một số lưu ý khi sử dụng số nguyên :

- Số nguyên không thể lưu được phần thập phân
- Số nguyên sẽ bị tràn giá trị nếu bạn gán cho nó giá trị vượt phạm vi của nó có thể lưu
- Thông thường bạn chỉ sử dụng int và long long, ít khi sử dụng short.
- Nếu lưu số nguyên từ -2.10^9 tới 2.10^9 thì dùng int là đủ, nếu ngoài phạm vi này dùng long long

2. Kiểu số thực

Khi bạn muốn lưu kiểu dữ liệu có phần thập phân bạn cần sử dụng kiểu dữ liệu số thực, trong C++ có 3 kiểu dữ liệu lưu số thực gồm : float, double, long double

Khi sử dụng kiểu dữ liệu số thực bạn cần biết các kiểu dữ liệu này có thể lưu được bao nhiêu số sau dấu phảy thập phân để lựa chọn kiểu dữ liệu cho phù hợp với độ chính xác mong muốn của bài toán.

Kiểu dữ liệu	Kích thước (byte)	Độ chính xác
float	4	6 - 9 chữ số thập phân
double	8	15 - 18 chữ số thập phân
long double	16	33 - 36 chữ số thập phân

Kiểu dữ liệu số thực có thể lưu được số nguyên tuy nhiên bạn không nên sử dụng nó để lưu số nguyên vì bản chất số thực luôn có sai số nhất định và không thể lưu giá trị số thực đúng tuyệt đối 100%. Thông thường thì mình sẽ dùng số thực double trong các bài toán khi kết quả cần lưu phần thập phân.

3. Kiểu dữ liệu kí tự

Khi bạn cần lưu trữ một ký tự trong C++ thì bạn sử dụng kiểu dữ liệu char, thực tế char cũng được sử dụng để lưu số nguyên.

Các ký tự như : chữ cái, chữ số, kí tự đặc biệt... đều có thể lưu trữ bằng kiểu dữ liệu char.

Kiểu dữ liệu	Kích thước (byte)	Giá trị có thể lưu	Phạm vi giá trị có thể lưu
char	1	Số nguyên hoặc ký tự	-128 -> 127
unsigned char	1	Số nguyên không âm hoặc ký tự	0 -> 255

4. Kiểu đúng sai

Kiểu đúng sai hay luận lý được sử dụng để lưu trữ giá trị đúng (true) hoặc sai (false)

Trong ngôn ngữ lập trình C++ thì kiểu bool được sử dụng để lưu các giá trị đúng sai.

Kiểu dữ liệu này có kích thước là 1 byte.

[*] BAI_3.KIEU_DU_LIEU_CO_BAN.cpp

```cpp
#include <iostream>
using namespace std;
int main()
{
    //1.Kieu so nguyen
    //nhap vao 2 so nguyen a, b.tinh tong va tich 2 so do
    int a,b;
    cout <<"Nhap so a = ";  cin >>a;
    cout <<"Nhap so b = ";  cin >>b;
    cout <<"a+b = "<<a+b <<" va a*b = "<<a*b <<endl;
    //2.kieu so thuc
    //nhap vao dtb hk1, dtb hk2. tinh dtb ca nam
    float tbk1,tbk2;
    cout <<"Nhap vao diem tb hk1 = ";   cin >>tbk1;
    cout <<"Nhap vao diem tb hk2 = ";   cin >>tbk2;
    float tbcn;
    tbcn=(tbk1 + tbk2*2)/3;
    cout <<"Diem trung binh ca nam cua ban = "<<tbcn<<endl;
    //3.Kieu ki tu
    // luu ten
    string Ten;
    cout <<"Nhap vao ten cua ban: ";    cin >>Ten;
    cout <<" Xin chao "<<Ten;
    //4. Kieu dung sai
    cout <<(a>b);
    return 0;
}
```

e

BÀI 4. BIẾN VÀ HẰNG TRONG C++

1. Biến là gì? Cách khai báo biến trong C++

Biến (variable) đại diện cho **vùng nhớ** lưu trữ dữ liệu (trên RAM) của chương trình. Biến chứa giá trị nhập vào, giá trị của một biểu thức, giá trị tính toán hoặc xử lý trong chương trình. Biến được nhận dạng thông qua **tên biến và kiểu dữ liệu**.

Biến được sử dụng như thế nào?

Biến cần được khai báo trước khi dùng (đọc/ghi giá trị). Chương trình tự động cấp phát vùng nhớ khi một biến được khai báo.

Cú pháp khai báo biến:

<Kiểu dữ liệu> *<Tên biến>;* Ví dụ: int a=10;

hoặc

<Kiểu dữ liệu> *<Tên biến 1>, <Tên biến 2>, ...;* Ví dụ: int a,b,c;

Khi khai báo biến cần xác định:

Biến lưu trữ dữ liệu gì -> xác định kiểu của biến

Ý nghĩa của biến -> xác định tên sẽ đặt cho biến

Ví dụ khi giải phương trình bậc 2: ax2 + bx + c = 0:

Lưu trữ hệ số: Kiểu float hoặc double, tại sao? Tên biến a, b, c, tại sao?

Lưu trữ delta: Kiểu float hoặc double, tại sao? Tên biến delta?

Lưu trữ nghiệm: Kiểu float hoặc double, tại sao? Tên biến x1, x2, s1, s2, so1, so2,…tại sao?

Quy tắc đặt tên biến

Không được trùng tên với từ khoá

Ký tự đầu: Một chữ (a, A, b, B, …) hay gạch dưới "_"

Không có ký hiệu đặc biệt nào khác: !,@,#,$,%,^,&,*,(,),…

Các ký tự tiếp theo: chữ, số, gạch dưới và không có khoảng trắng

Khởi tạo giá trị cho biến

Các biến có thể được khởi tạo một giá trị khi khai báo hoặc gán giá trị sau khi khai báo. Giá trị các biến cũng có thể thay đổi. **Các giá trị được lưu trong vùng nhớ của biến**.

Ví dụ: int a=0, b=5;

Giá trị khởi tạo mặc định của biến

Khi khai báo mà không khởi tạo giá trị cho biến, biến sẽ nhận các giá trị mặc định được quy định bởi một số ít trình biên dịch C/C++.

– Các biến số được khởi tạo giá trị mặc định là 0, 0.0 hoặc 0.0000000000.

– Các biến ký tự có giá trị khởi tạo là null ('\0')

– Các biến boolean có trị khởi khởi tạo là false (0).

Lưu ý:

Không phải trình biên dịch C/C++ nào cũng chấp nhận không khởi tạo giá trị.

Một số trình biên dịch sẽ báo lỗi hoặc gán giá trị rác cho biến, dẫn đến kết quả chương trình sai nghiêm trọng. Tốt nhất bạn nên khởi tạo giá trị cho biến trước khi sử dụng.

2. Tầm vực của biến

Còn gọi là **phạm vi** hay **scope** của biến. Tầm vực của biến là vùng chương trình mà một biến tồn tại và sử dụng được.

Các loại phạm vi của biến

– Toàn cục (global): bên ngoài tất cả các hàm, được sử dụng trong toàn chương trình.

– Cục bộ (local)**: chỉ sử dụng bên trong hàm.**

Thân hàm: từ dấu { đến dấu } của thân hàm

Hoặc các khối con (từ dấu { đến dấu } của khối)

3. Hằng (constant) là gì?

Vùng nhớ **có khả năng thay đổi hoặc không cho thay đổi dữ liệu**. **Các vùng nhớ** mà giá trị dữ liệu **có thể thay đổi** được gọi là **các biến**.

Các vùng nhớ mà giá trị dữ liệu được khởi tạo một lần và **không bao giờ thay đổi** được gọi là **các hằng số**. Chúng thường được định nghĩa trong vùng toàn cục và được đặt trước bởi từ khóa **const** hoặc thông qua cú pháp **#define**.

Cú pháp khai báo hằng:

#define <Tên hằng> **<Giá trị>**

const **<Kiểu dữ liệu>** <Tên hằng> **= <Giá trị>;**

Ví dụ, có thể khai báo hằng PI như sau:

```
const float PI = 3.14;
//hoặc
#define PI 3.14
```

4. TỪ KHÓA (KEYWORD)

Keywords - Từ khóa được định nghĩa sẵn trong ngôn ngữ lập trình C++, bạn sẽ gặp rất nhiều từ khóa như : cout, cin, int, float, using, for, while, try...

Bảng sau liệt kê các từ khóa trong ngôn ngữ lập trình C++

Bảng sau liệt kê các từ khóa trong ngôn ngữ lập trình C++

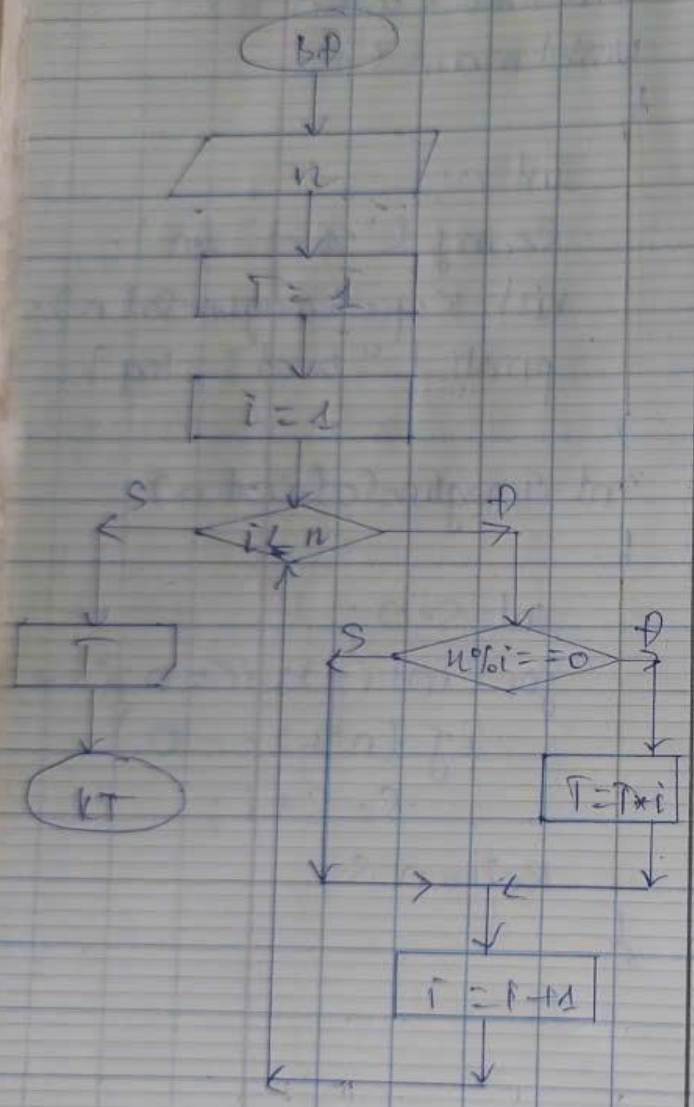
A-C	D-P	R-Z
	ecltype (C++11)	reflexpr (reflection TS)
alignas (C++11)	default	register (2)
alignof (C++11)	delete	reinterpret_cast
and	do	requires (C++20)
and_eq	double	return
asm	dynamic_cast	short
atomic_cancel	else	signed
atomic_commit	enum	sizeof (1)
atomic_noexcept	explicit	static
auto	export	static_assert (C++11)
bitand	extern	static_cast
bitor	false	struct (1)
bool	float	switch
break	for	synchronized (TM TS)
case	friend	template
catch	goto	this (4)
char	if	thread_local (C++11)
char8_t (C++20)	inline	throw
char16_t (C++11)	int	true
char32_t (C++11)	long	try
class	mutable	typedef
compl	namespace	typeid
concept (C++20)	new	typename
const	noexcept (C++11)	union
consteval (C++20)	not	unsigned
constexpr (C++11)	not_eq	using (1)
constinit (C++20)	nullptr (C++11)	virtual
const_cast	operator	void
continue	or	volatile
co_await (C++20)	or_eq	wchar_t
co_return (C++20)	private	while
co_yield (C++20)	protected	xor
	public	xor_eq

[*] BAI_4_BIEN_HANG.cpp

```cpp
#include <iostream>
using namespace std;
const float PI=3.14;//Bien toan cuc
int main()
{
    //viet chuong trinh tinh dien tich hinh tron.
    float r=0.0,s=0.0;
    cout <<"Ban hay nhap ban kinh hinh tron = ";
    cin >>r;
    s=r*r*PI;
    cout <<"Dien tich hinh tron = " <<s;
    return 0;
}
```

Bài 16: Tính tích tất cả "ước số" của số nguyên dương n.

Bài 22: Tính tích tất cả "ước số" của số nguyên dương n.

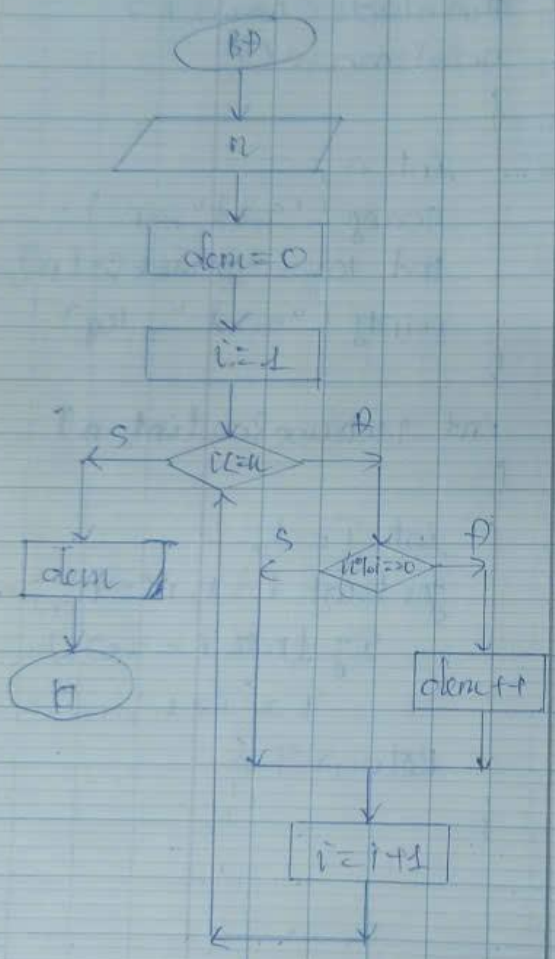


Chạy từng bước với n = 4

```
n = 4
T = 1
i = 1
while (1 <= 4) Đ
    If (4%1 == 0) Đ
        T = 1
    i = 2
while (2 <= 4) Đ
    If (4%2 == 0) Đ
        T = 1*2 = 2
    i = 3
while (3 <= 4) Đ
    If (4%3 == 0) S
    i = 4
while (4 <= 4) Đ
    If (4%4 == 0) Đ
        T = 2*4 = 8
    i = 5
while (5 <= 4) S
    Xuất (T)
```

Bài 17: Đếm số lượng "ước số" của số nguyên dương n.

Bài 23: Đếm số lượng "ước số" của số nguyên dương n



Chạy từng bước với n=4

```
n=4
dem=0
i=1
while(1<=4) Đ
        if(4%1==0) Đ
            dem=1
        i=2
while(2<=4) Đ
        if(4%2==0) Đ
            dem=2
        i=3
while(3<=4) Đ
        if(4%3==0) S
        i=4
while(4<=4) Đ
        if(4%4==0) Đ
            dem=3
        i=5
while(5<=4) S
    Xuất (dem)
```

Thuật toán tối ưu bài 17.

Bằng cách phân tích N thành thừa số nguyên tố.

VD: Ước (60)=1,2,3,4,5,6,10,12,15,20,30,60.

Nếu N=I*j thì cả I và j đều là ước của N

Nếu N=I*j tất nhiên N chia hết cho cả I và j

=> với mỗi uoc I tim duoc thi co them 1 uoc nua chinh la j=n/I

Số chính phương là số mà có 1 số nào đó bình phương lên bằng chính nó.

Vd: 9 là số chính phương vì có 3*3=9

Vẽ lưu đồ

N=81 ước 1,3,9,27,81

I=1

81/1==0

    Dem=1;

    J=81

    J!=I đ dem=2

I=2

    81%2==0 s

I=3

    81%3==0

        Dem=3

        J=81/3=27

        27!=3 dem=4

I=4

81%4==0 sai

I=5 sai

I=6 sai

I=7 sai

I=8 sai

I=9

81%9==0 Đ

Dem=5

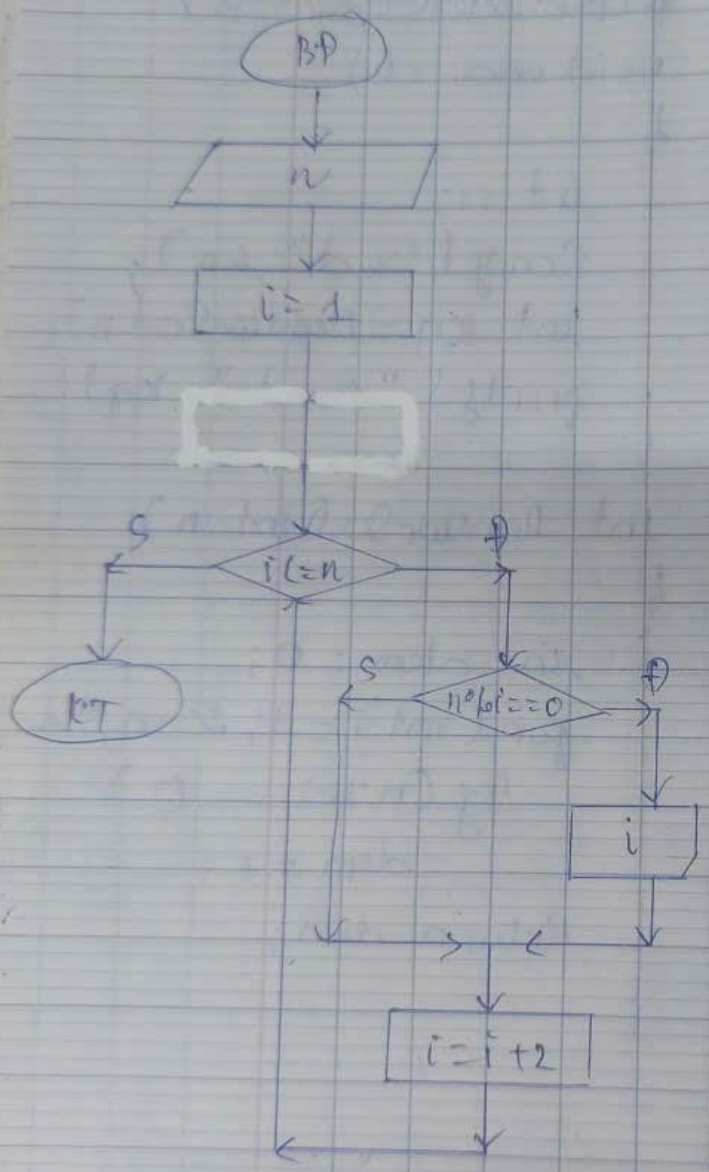
J=81/9= =9 sai

I=10

Return dem =5

Bài 18. Liệt kê tất cả “Ước số lẻ” của số nguyên dương n.

Bài 24: Liệt kê tất cả "ước số lẻ" của số nguyên dương n.

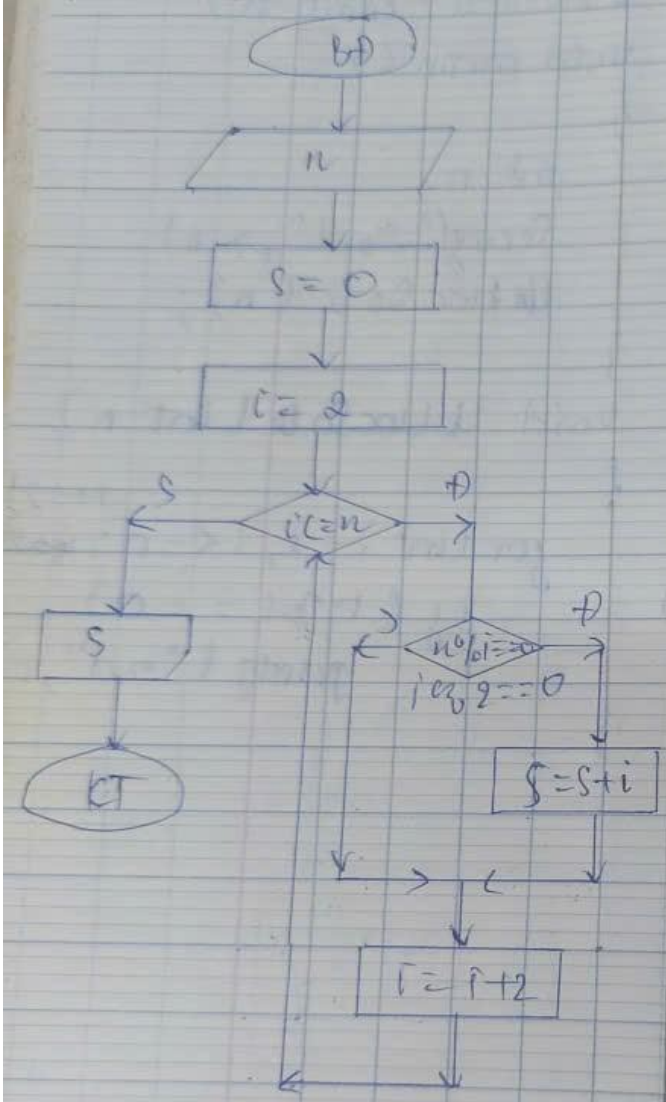


Chạy từng bước với n = 4

```
n = 4
i = 1
while (1 <= 4) Đ
    if (4 % 1 == 0) Đ
        Xuất (1)
    i = 3
while (3 <= 4) Đ
    if (4 % 3 == 0) S
    i = 5
while (5 <= 4) S.
```

Bài 19. Tính tổng tất cả “Ước số chẵn” của số nguyên dương n.

Bài 25: Tính tổng tất cả "ước số chẵn" của số nguyên dương

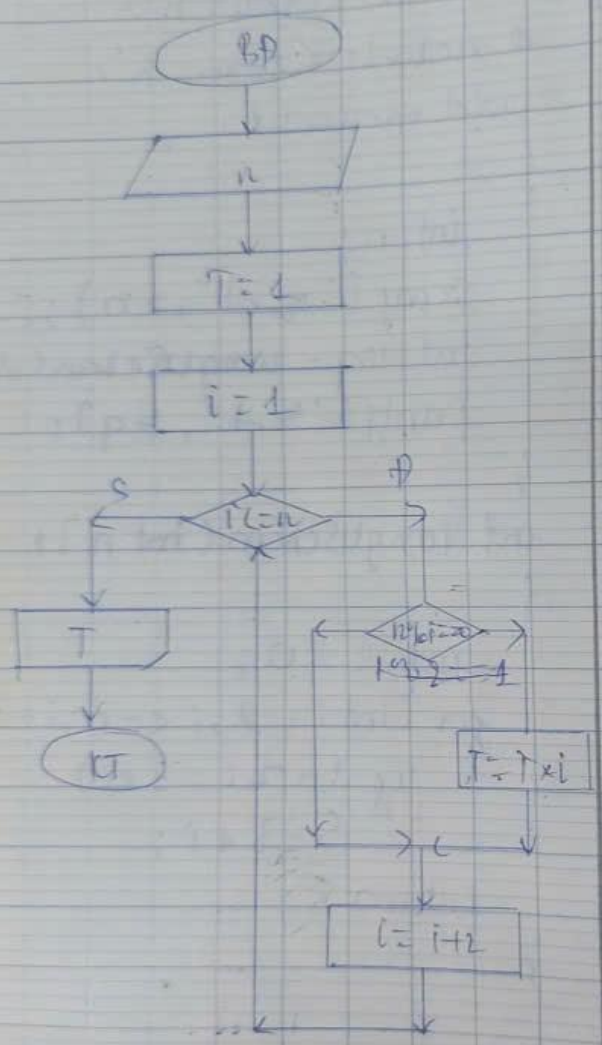


chạy từng bước với n=10

```
n=10
S=0
i=2
while (2 <= 10) Đ
        if (10 % 2 == 0) Đ
                S = 2
        i = 4
while (4 <= 10) Đ
        if (10 % 4 == 0) S
        i = 6
while (6 <= 10) Đ
        if (10 % 6 == 0) S
        i = 8
while (8 <= 10) Đ
        if (10 % 8 == 0) S
        i = 10
while (10 <= 10) Đ
        if (10 % 10 == 0) Đ
                S = 2 + 10 = 12
        i = 12
while (12 <= 10) S
        xuất (S)
```

Bài 20. Tính tích tất cả “Ước số lẻ” của số nguyên dương n.

Bài 26: Tính tích tất cả "ước số lẻ" của số nguyên dương

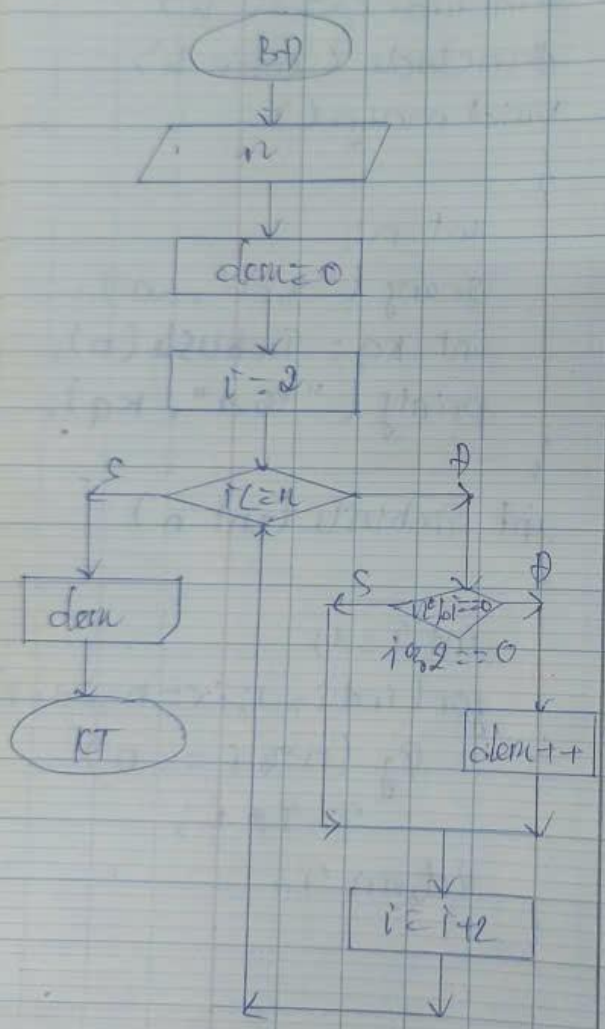


Chạy từng bước với n = 10

```
n = 10
T = 1
i = 1
while (1 <= 10) Đ
    If (10 % 1 == 0) Đ
        T = 1
    i = 3
while (3 <= 10) Đ
    If (10 % 3 == 0) S
    i = 5
while (5 <= 10) Đ
    If (10 % 5 == 0) Đ
        T = 5
    i = 7
while (7 <= 10) Đ
    If (10 % 7 == 0) S
    i = 9
while (9 <= 10) Đ
    If (10 % 9 == 0) S
    i = 11
while (11 <= 10) S
    xuất (T)
```

Bài 21. Đếm số lượng “Ước số chẵn” của số nguyên dương n.

Bài 27: Đếm số lượng "ước số chẵn" của số nguyên dương

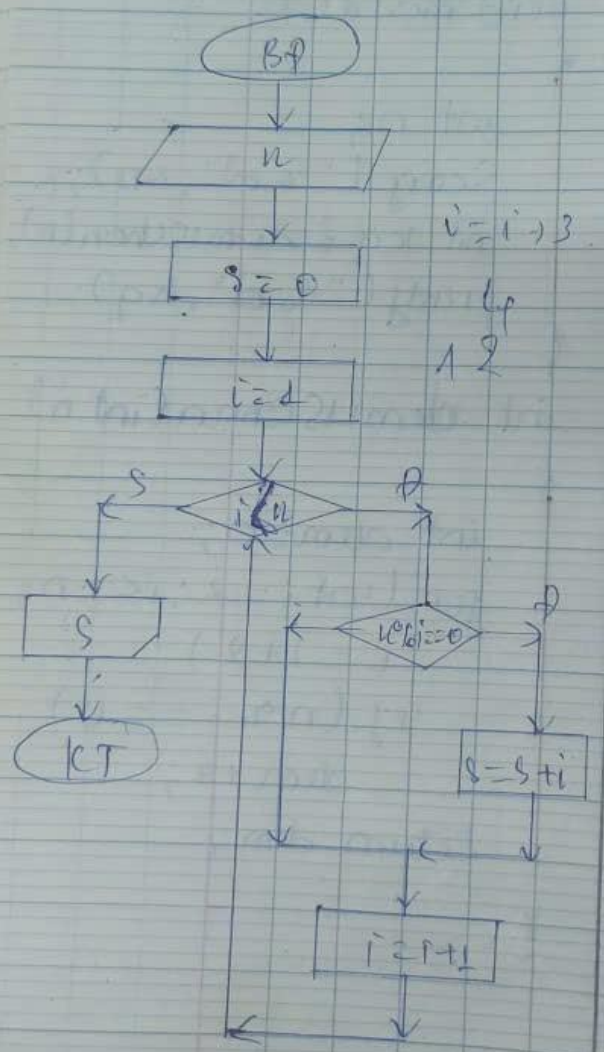


Chạy từng bước với n=10

```
n=10
dem=0
i=2
while(2<=10) Đ
    If(10%2==0) Đ
        dem=1
    i=4
while(4<=10) Đ
    If(10%4==0) S
    i=6
while(6<=10) Đ
    If(10%6==0) S
    i=8
while(8<=10) Đ
    If(10%8==0) S
    i=10
while(10<=10) Đ
    If(10%10==0) Đ
        dem=2
    i=12
while(12<
```

Bài 22. Cho số nguyên dương N. Tính tổng các ước số nhỏ hơn chính nó.

Bài 28: Cho số nguyên dương n. Tính tổng các ước số nhỏ hơn chính nó.



i = i + 3

Lp

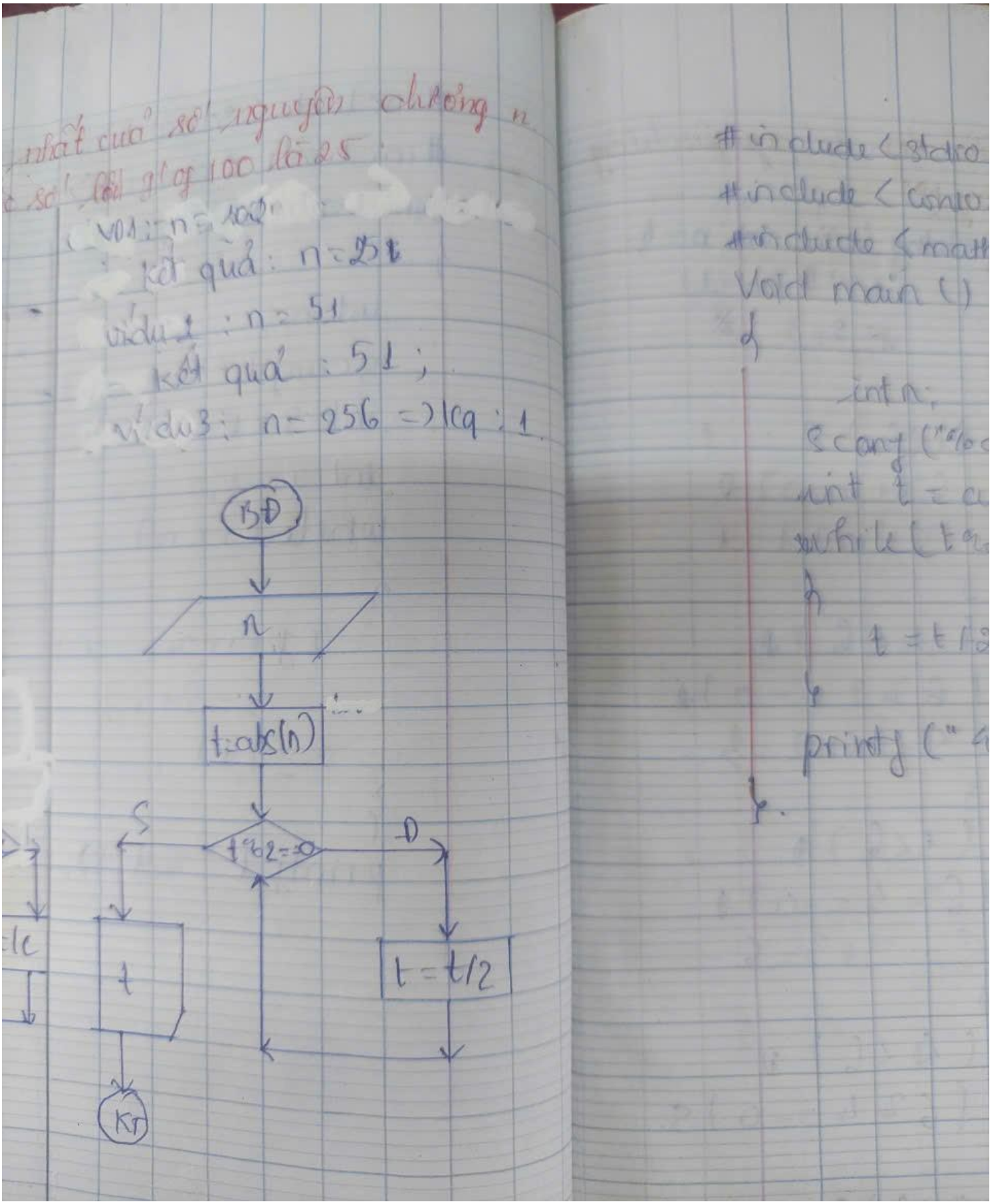
1 2

Chạy từng bước với n = 4

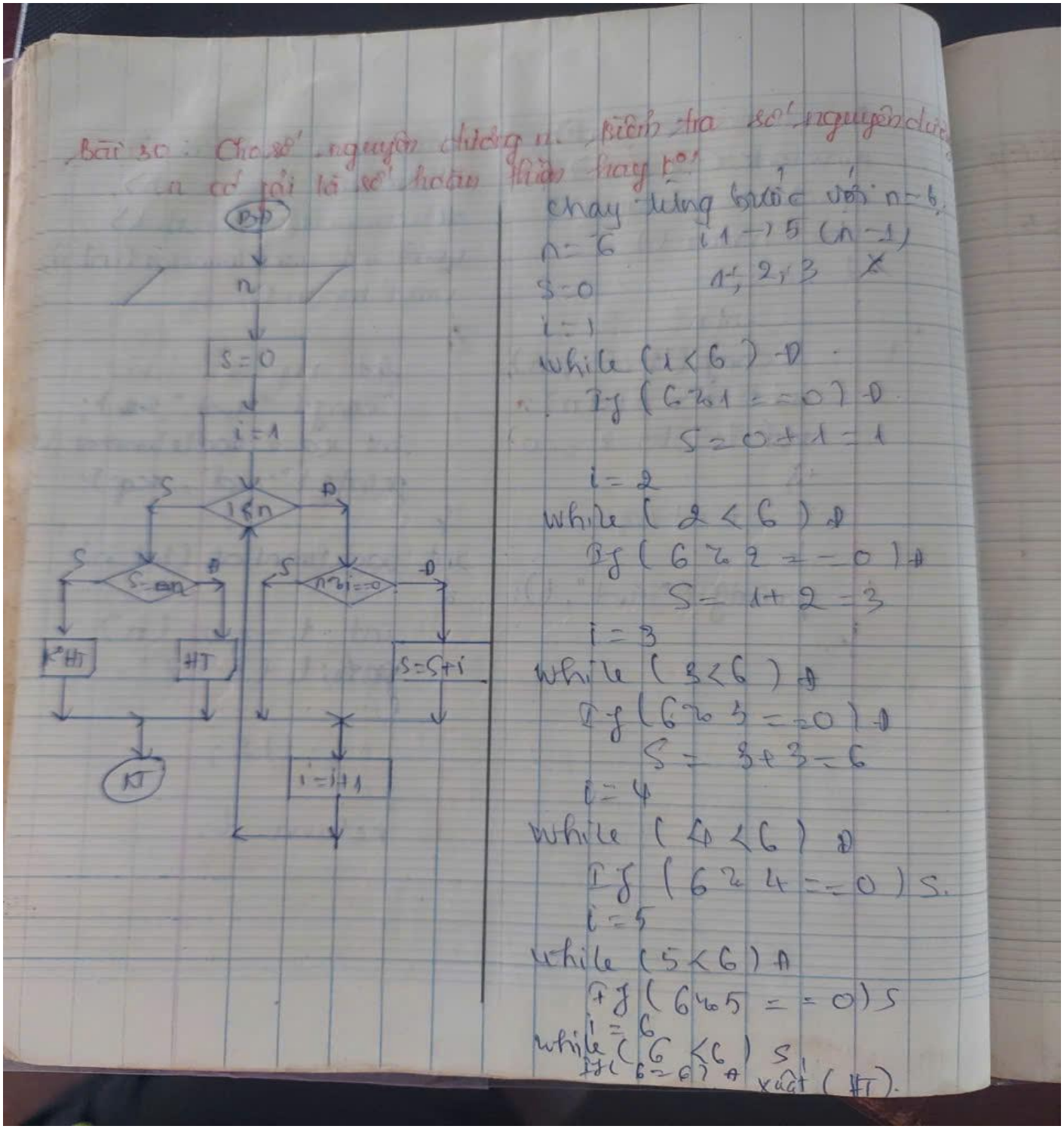
```
n = 4
S = 0
i = 1
while (1 < 4) Đ
        If (4 % 1 == 0) Đ
                S = 1
        i = 2
while (2 < 4) Đ
        If (4 % 2 == 0) Đ
                S = 3
        i = 3
while (3 < 4) Đ
        If (4 % 3 == 0) S
        i = 4
while (4 < 4) S
        Xuất (S).
```

Bài 23. Tìm ước số lẻ lớn nhất của số nguyên dương N.

Ví dụ: N=100 ước số lẻ lớn nhất của 100 là 25.

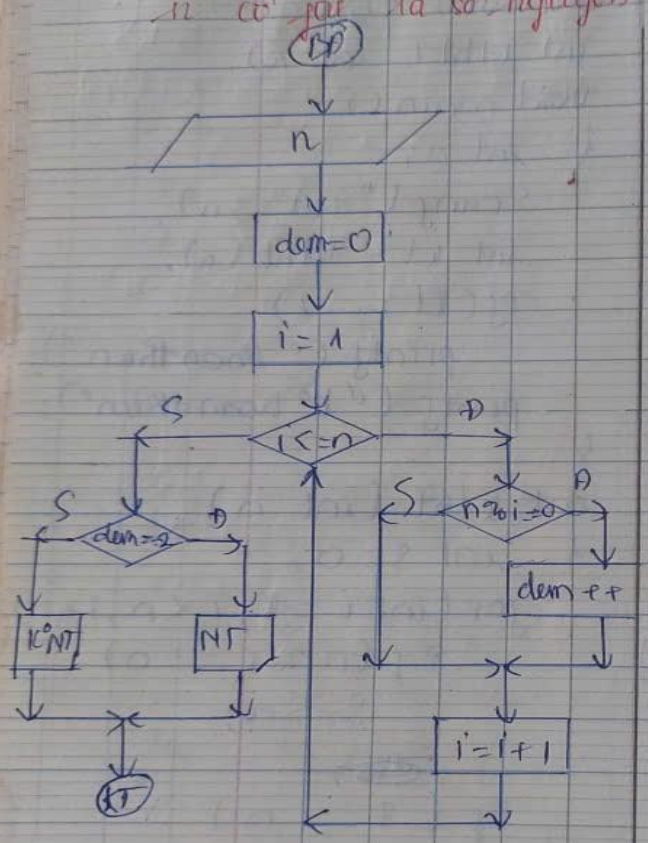


Bài 24. Cho số nguyên dương N. Kiểm tra N có phải là số hoàn thiện (số hoàn hảo - perfect numbers) hay không?



Bài 25. Cho số nguyên dương N. Kiểm tra N có phải là số nguyên tố hay không?

Bài 31: Cho số nguyên dương n. Kiểm tra số nguyên dương n có phải là số nguyên tố hay k?

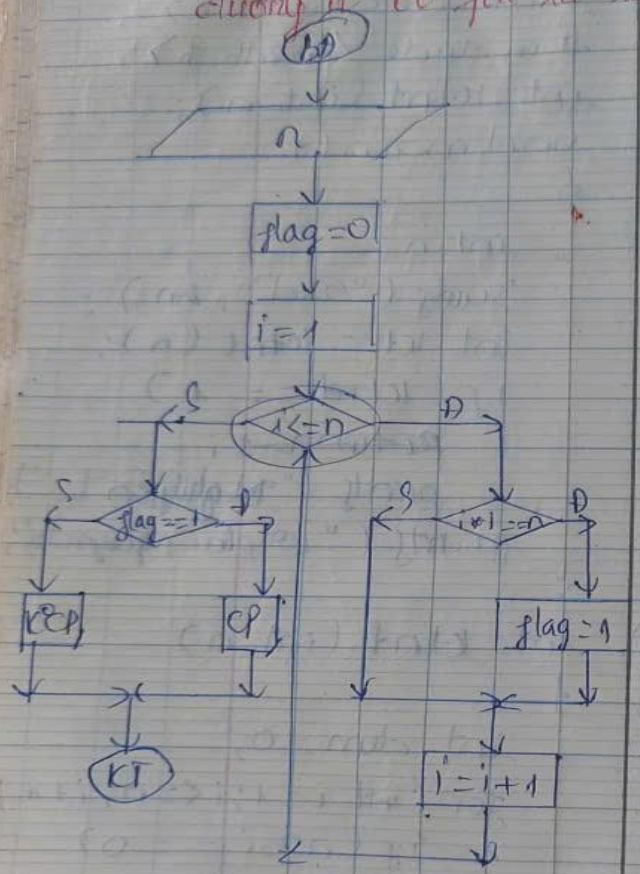


chạy từng bước với n=5

```
n=5
dem=0
i=1
while (1<=5) Đ
    if (5%1==0) Đ
        dem=1
    i=2
while (2<=5) Đ
    if (5%2==0) S
    i=3
while (3<=5) Đ
    if (5%3==0) S
    i=4
while (4<=5) Đ
    if (5%4==0) S
    i=5
while (5<=5) Đ
    if (5%5==0) Đ
        dem=2
i=6
while (6<=5) S
    if (dem==2) Đ
        xuat(NT)
```

Bài 26. Cho số nguyên dương N. Kiểm tra N có phải là số chính phương hay không?

Bài 32: Cho số nguyên dương n. Kiểm tra số nguyên dương n có phải là số chính phương hay k°?



chạy từng bước với n=4

```
n=4
flag=0
i=1
while (1<=4) Đ
    if (1*1==4) S.
    i=2
while (2<=4) Đ
    if (2*2==4) Đ
        flag=1
    i=3.
while (3<=4) Đ
    if (3*3==4) S
    i=4
while (4<=4) Đ
    if (4*4==4) S
    i=5
while (5<=4) S
    if (flag==1) Đ
        xuất (CP).
```

Bài 27 (btvn). Cho hai số nguyên dương N và M. Kiểm tra hai số đó có phải là số bạn bè – friend numbers hay không?

Các số bè bạn n, m là hai số nguyên mà tổng ước số của n bằng m và tổng các ước của m bằng n, ví dụ như cặp số bè bạn nhỏ nhất là 220 và 284.

Bài 28. [Đọc và xuất tệp] Đếm số lượng chữ số của số nguyên dương n.

N= 3578925

KĨ THUẬT TÁCH CHỮ SỐ.

N=35682 /10 = 3568 DƯ 2

35682 % 10 = 2

N=N/10 = 35682 / 10 = 3568.

3568/10=356

356/10=35

35/10=3

3/10 =0

Bài 29. [Đọc và xuất tệp] Tính tổng các chữ số của số nguyên dương n.

Bài 30. [Đọc và xuất tệp] Tính tích các chữ số của số nguyên dương n.

Bài 31. [Đọc và xuất tệp] Hãy đếm số lượng chữ số lẻ của số nguyên dương n.

Bài 32. [Đọc và xuất tệp] Hãy tính tổng các chữ số chẵn của số nguyên dương n.

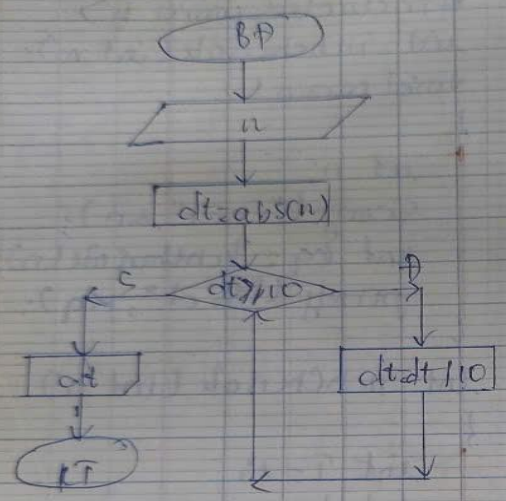
Bài 33. [Đọc và xuất tệp] Hãy tính tích các chữ số lẻ của số nguyên dương n.

Bài 34. [Đọc và xuất tệp] Hãy tìm chữ số đầu tiên của số nguyên dương n.

VD: N= 35269

N = N/10

Bài 49: Cho số nguyên dương n. Hãy tìm chữ số đầu tiên của n.



Chạy từng bước với n = 3579

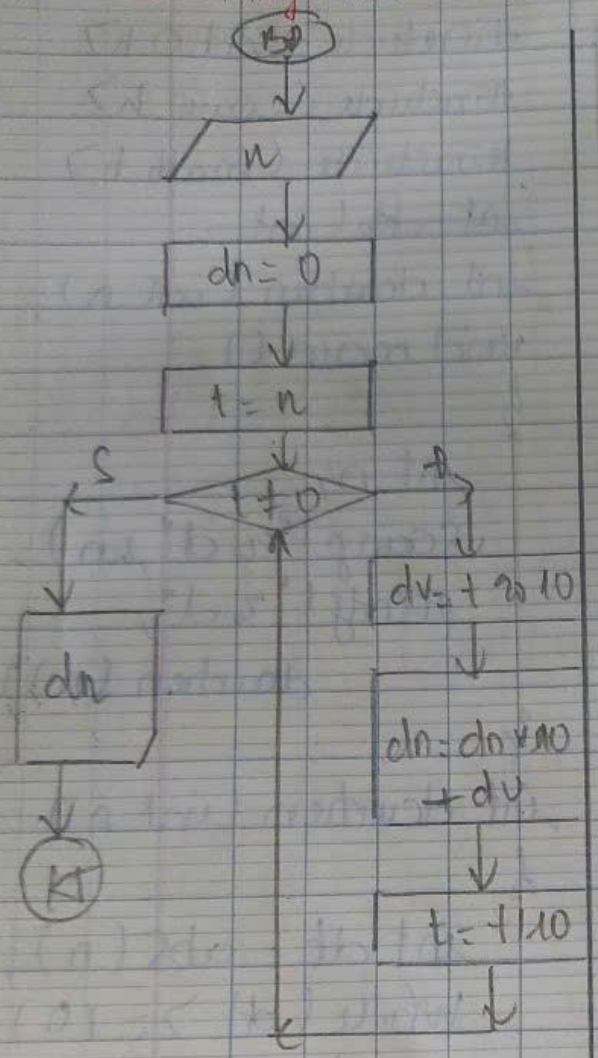
```
dt = abs(3579)
while (3579 > 10) Đ
    dt = 357
while (357 > = 10) Đ
    dt = 35
while (35 > = 10) Đ
    dt = 3
while (3 > = 10) S
    xuat (dt)
```

Bài 35. [Đọc và xuất tệp] Hãy tìm chữ số đảo ngược của số nguyên dương n.

VD: N= 35269

Số đảo ngược của N = 96253

Bài 50: Hãy tìm chữ số đảo ngược của số nguyên dg n.



Chạy từng bước với n = 365

n = 365
dn = 0
t = 365 — 365 ≠ 0 Đ
dv = 5
dn = 0 * 10 + 5 = 5
t = 365 / 10 = 36 — 36 ≠ 0 Đ
dv = 6
dn = 5 * 10 + 6 = 56
t = 36 / 10 = 3 — 3 ≠ 0 Đ
dv = 3
dn = 56 * 10 + 3 = 563
t = 3 / 10 = 0 — 0 ≠ 0 S

xuất (dn)

KĨ THUẬT ĐẶT LÍNH CANH

Bài 36. [Đọc và xuất tệp] Hãy tìm chữ số lớn nhất của số nguyên dương n.

VD: N= 7526

Lc = 3526 %10 = 6

While (7256 !=0) Đ

    Dv=7256%10=6

    If (6>6)Sai

    N=725

While (725 !=0) Đ

    Dv=725%10=5

    If (5>6)Sai

    N=72

While (72 !=0) Đ

    Dv=72%10=2

    If (2>6)Sai

    N=7

While (7 !=0) Đ

    Dv=7%10=7

    If (7>6) Đúng

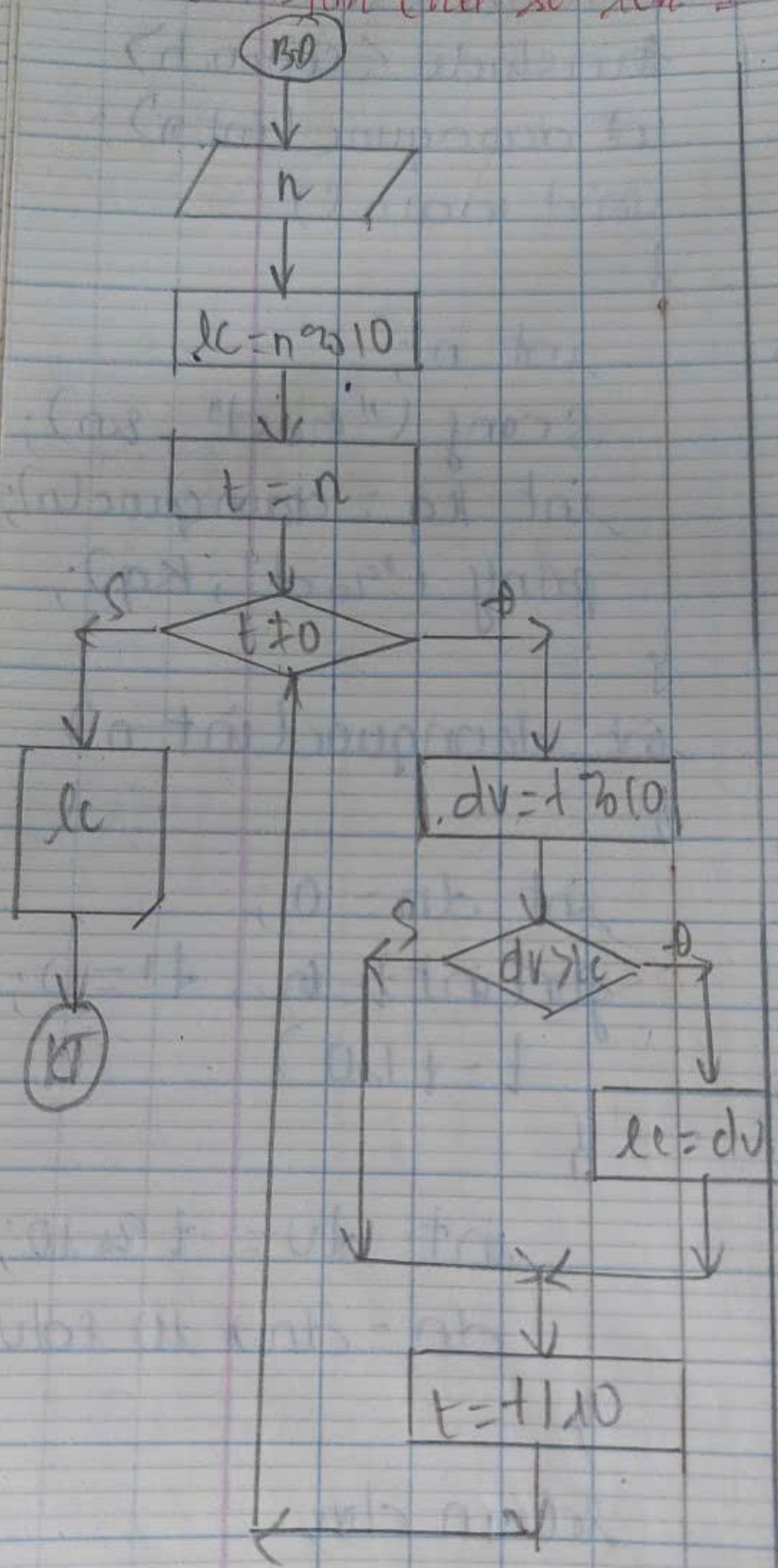
        Lc=7;

    N=0

While (0 !=0) sai

XUẤT LC = 7

Bài 5.1. Tìm chiều số đối xứng của số nguyên



n = 5 2

lc = n

tam = n

Bài 37. [Đọc và xuất tệp] Hãy tìm chữ số nhỏ nhất của số nguyên dương n.

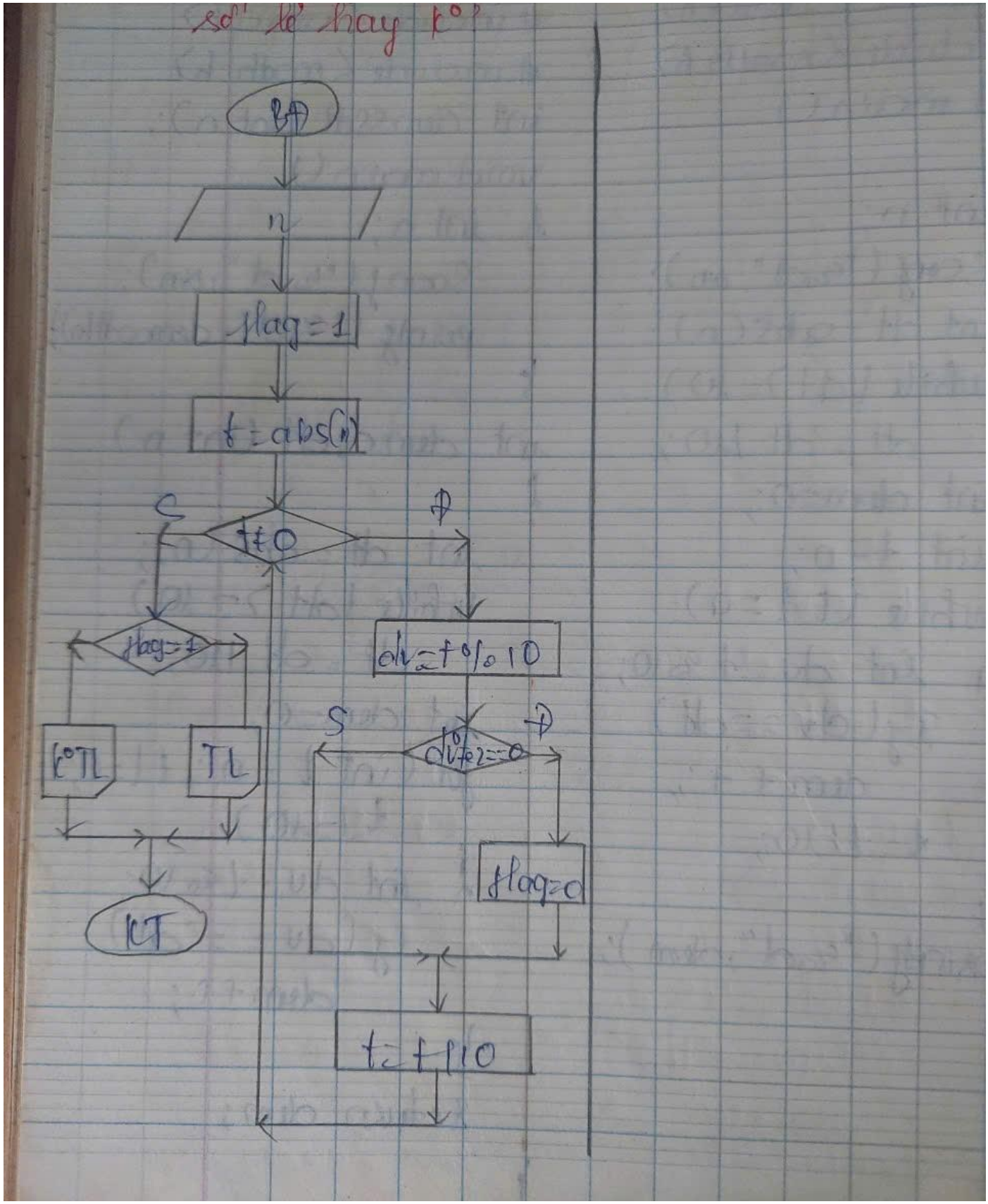
Bài 38. [Đọc và xuất tệp] Hãy đếm số lượng chữ số lớn nhất của số nguyên dương n.

Bài 39. [Đọc và xuất tệp] Hãy đếm số lượng chữ số nhỏ nhất của số nguyên dương n.

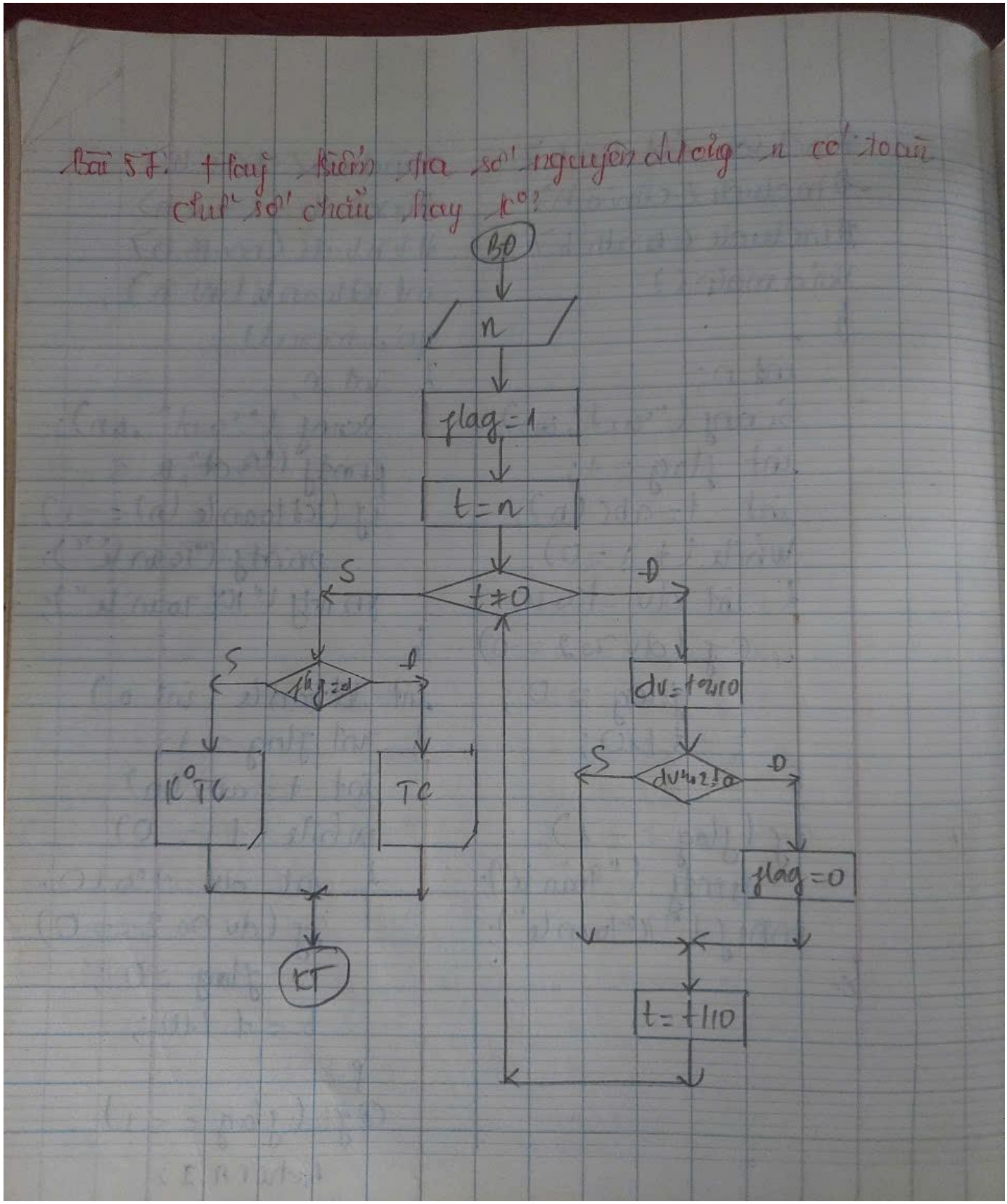
Bài 40. [Đọc và xuất tệp] Hãy đếm số lượng chữ số đầu tiên của số nguyên dương n.

KĨ THUẬT ĐẶT CỜ HIỆU

Bài 41. [Đọc và xuất tệp] Hãy kiểm tra số nguyên dương n có toàn chữ số lẻ hay không?



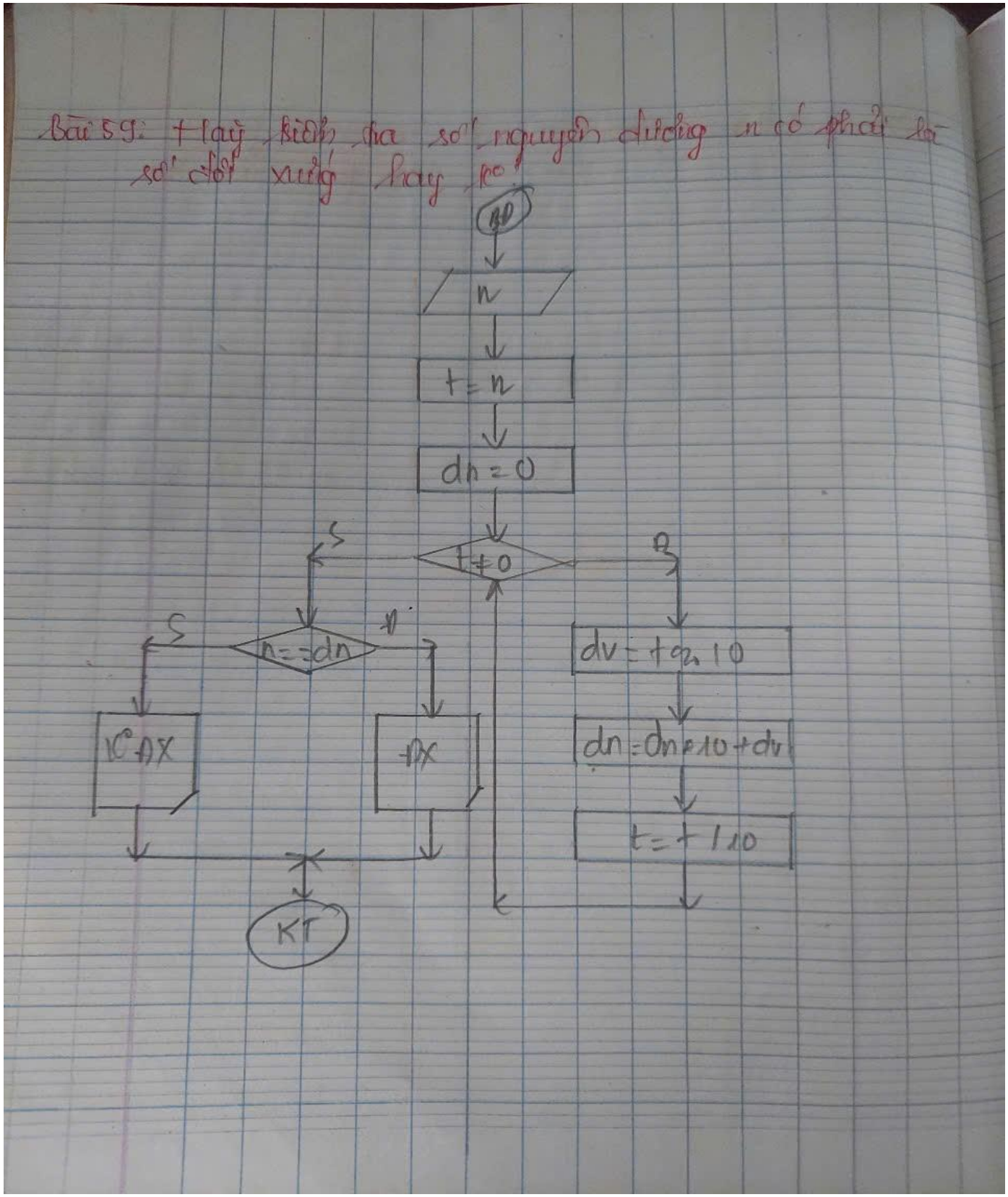
Bài 42. [Đọc và xuất tệp] Hãy kiểm tra số nguyên dương n có toàn chữ số chẵn hay không?



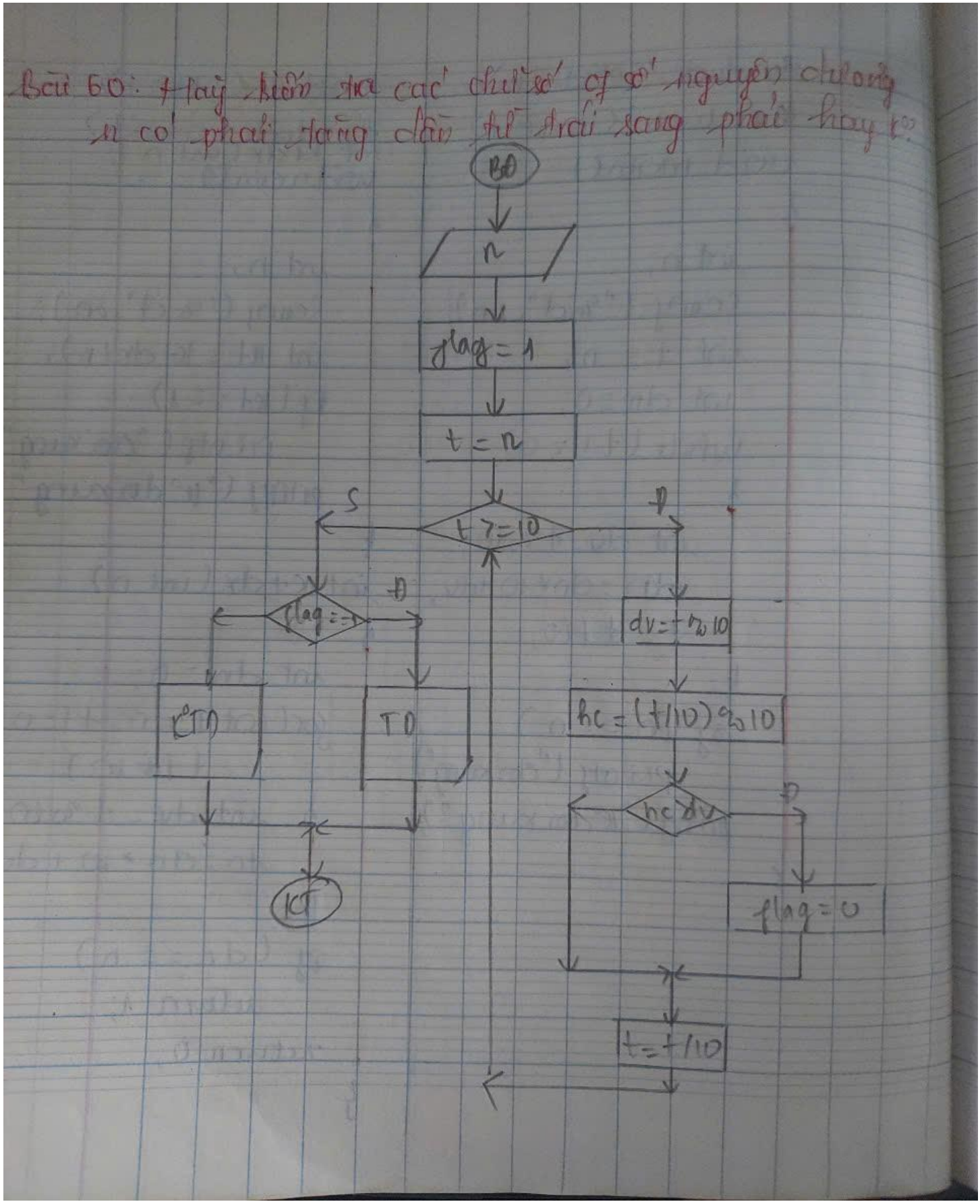
Bài 43. [Đọc và xuất tệp] Hãy kiểm tra số nguyên dương n có phải là số đối xứng hay không?

VD: N= 34725 DN: 52743 KO PHẢI SỐ ĐX

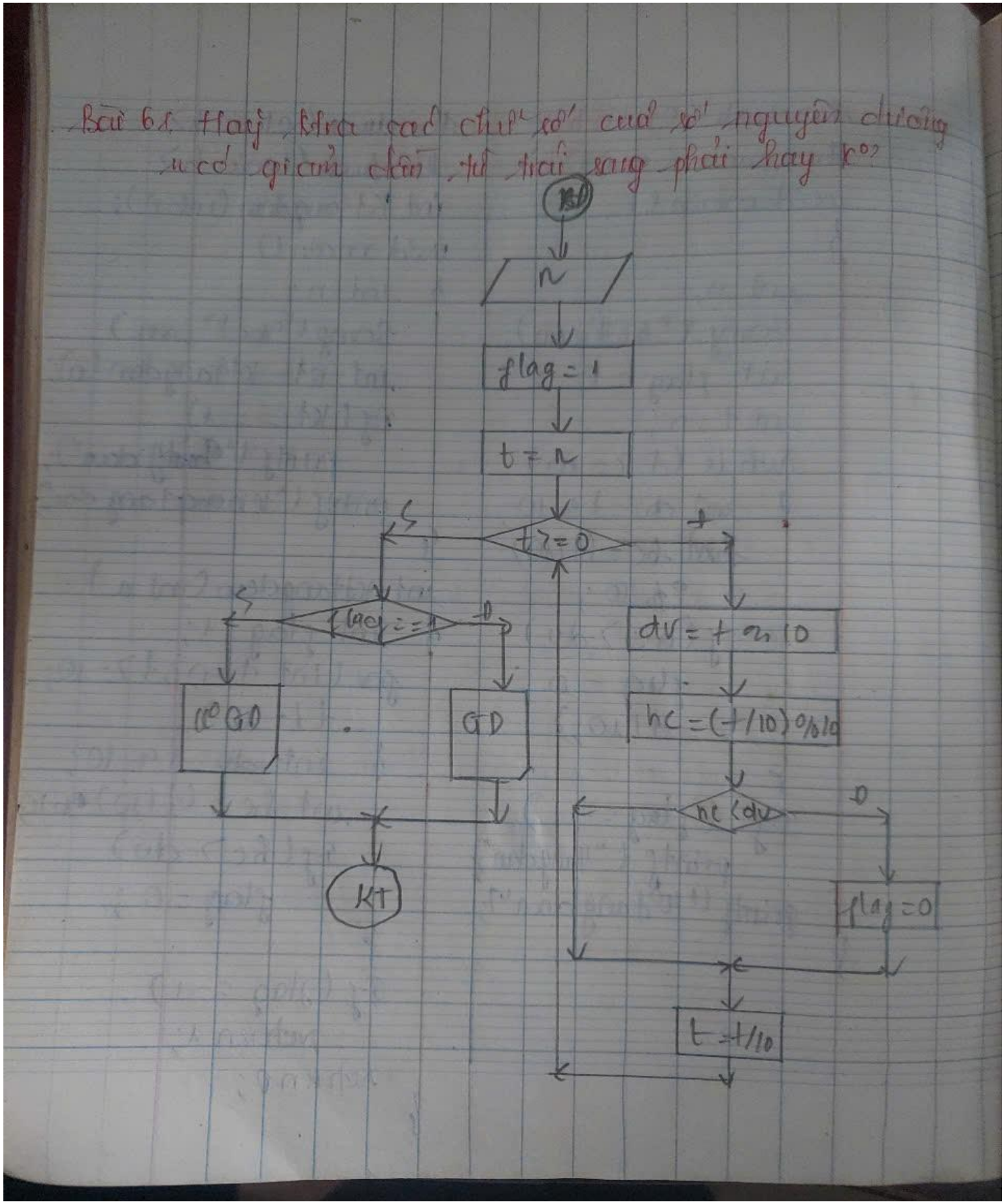
N=2345432 DN: 2345432 SỐ ĐỐI XỨNG



Bài 44. [Đọc và xuất tệp] Hãy kiểm tra số nguyên dương n có tăng dần từ trái sang phải hay không?

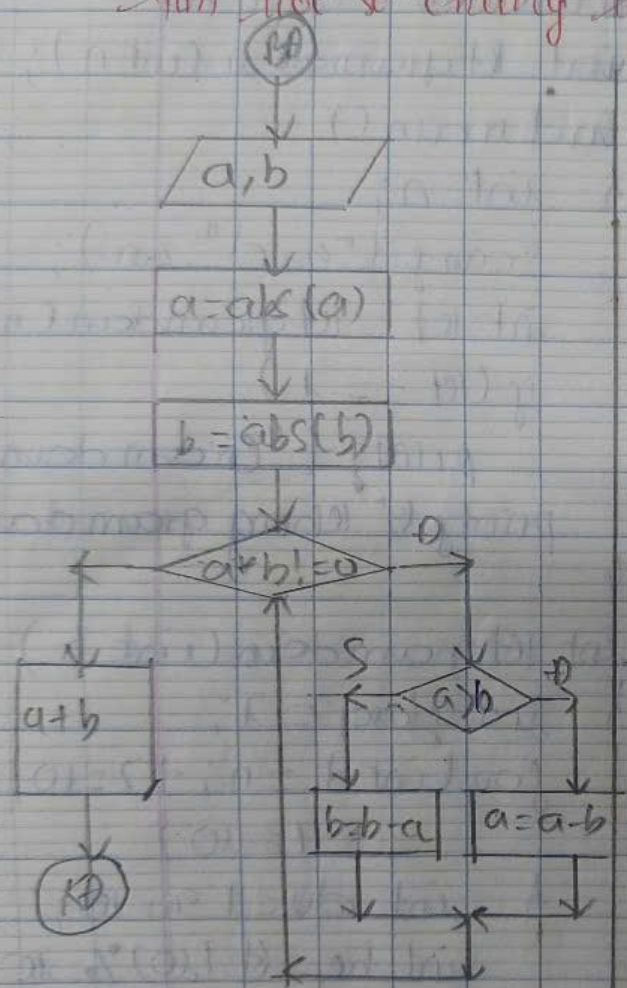


Bài 45. [Đọc và xuất tệp] Hãy kiểm tra số nguyên dương n có giảm dần từ trái sang phải hay không?



Bài 48: Cho 2 số nguyên dương a và b. Hãy viết chương trình tìm UCLN.

Bài 62: Cho 2 số nguyên dương a & b. Hãy viết lưu đồ tìm ước số chung lớn nhất của 2 giá trị này.



Chạy từng bước với a=6, b=8.

a = abs(a) = 6

b = abs(b) = 8

6*8 ≠ 0 Đ

If (6 > 8) S

b = 8 - 6 = 2

6*2 ≠ 0 Đ

If (6 > 2) Đ

a = 6 - 2 = 4

(4*2) = 0 Đ

If (4 > 2) Đ

a = 4 - 2 = 2

2*2 = 0 Đ

If (2 > 2) S

b = 2 - 2 = 0

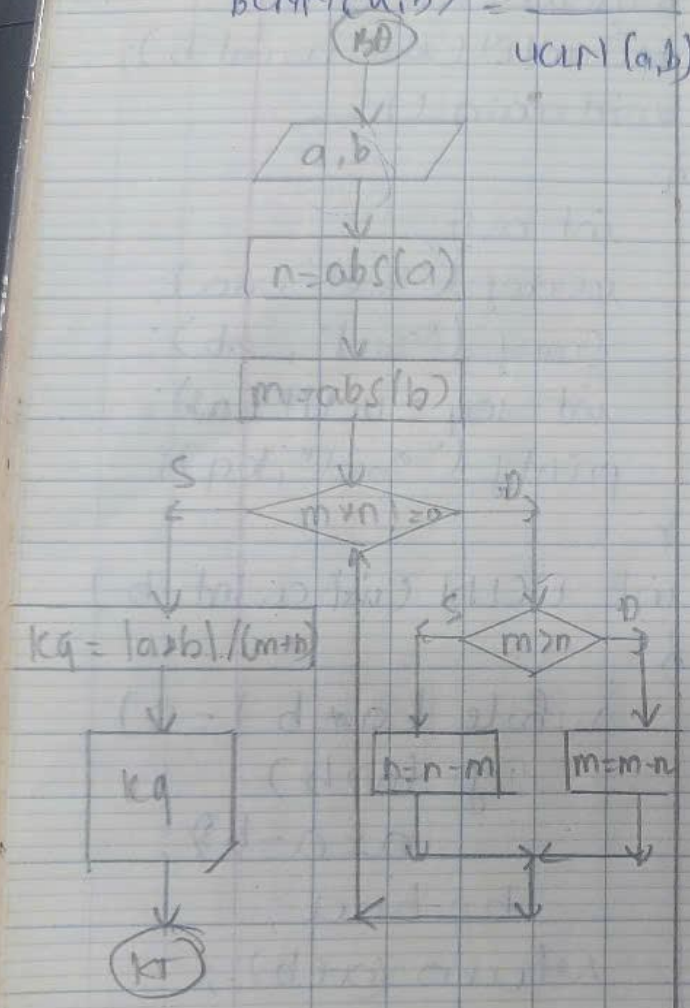
2*0 = 0 S

Xuất (2+0)

Bài 49: Cho 2 số nguyên dương a và b. Hãy viết chương trình tìm BCNN.

Bài 63 Cho 2 số nguyên dương a và b. Hãy vẽ lưu đồ tìm bội chung nhỏ nhất của hai giá trị này.

$$BCNN(a,b) = \frac{|a*b|}{UCLN(a,b)}$$



Chạy từng bước với a=6, b=8

n = abs(a) = 6.

m = abs(b) = 8 — 6*8 ≠ 0 Đ

If (8 > 6) Đ

m = 8 - 6 = 2 — 6*2 ≠ 0 Đ

If (2 > 6) S

n = 6 - 2 = 4 — 4*2 ≠ 0 Đ

If (2 > 4) S

n = 4 - 2 = 2 — 2*2 ≠ 0 Đ

If (2 > 2) S

n = 2 - 2 = 0 — 2*2 ≠ 0 S

Kq = |6*8|/(2+0)

= 24

Xuất (Kq).

TỐI ƯU THUẬT TOÁN TÌM ƯỚC CHUNG LỚN NHẤT

Vd 1: Tìm UCLN của 8 và 6

Gọi r là số dư của a/b

Bước	a	b	r
1	8	6	2
2	6	2	0
3	2	0	-

Vd 2: Tìm UCLN của 6 và 8 (a<b)

Nếu a<b thì b là số dư của b/a

Bước	a	b	r
1	6	8	2
2	6	2	0
3	2	0	-

Vd 3: Tìm UCLN của 13 và 111 (a<b)

Nếu a<b thì b là số dư của b/a

Bước	a	b	r
1	13	7(111%13)	6
2	7	6	1
3	6	1	0
4	1	0	

Vd 4: Tìm UCLN của 136 và 111 (a<b)

Nếu a<b thì b là số dư của b/a

Bước	a	b	r
1	136	111	25
2	111	25	11
3	25	11	3
4	11	3	2
5	3	2	1
6	2	1	0
7	1	0	-

Vd 5: Tìm UCLN của 221 và 111 (a<b)

Nếu a<b thì b là số dư của b/a

Bước	a	b	r
1	221	111	110
2	111	110	1
3	110	1	0
4			
5			
6			
7			

HƯỚNG DẪN GIẢI MỘT SỐ BÀI TẬP CƠ BẢN TRONG luyencode.net

1. VL07 - Tính tổ hợp

☺ VL07 - Tính tổ hợp

Viết chương trình tính tổ hợp chập k của n phần tử có công thức như dưới đây:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

Giới hạn

- $1 \leq k \leq n \leq 25$

Input

Lần lượt là 2 số n, k cách nhau bởi khoảng trắng

Output

Kết quảtổ hợp chập k của n

Tổ hợp là gì?

Tổ hợp là cách **chọn ra k phần tử từ n phần tử ban đầu**, **không quan tâm đến thứ tự**.

Ví dụ: Từ 5 học sinh A, B, C, D, E bạn chọn ra 2 bạn để đi trực nhật.

Các tổ hợp gồm: AB, AC, AD, AE, BC, BD, BE, CD, CE, DE.

Tổng cộng có **10 cách chọn**.

Công thức tính tổ hợp:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

n! là giai thừa của n: n!=n×(n−1)×…×1

Ví dụ:

5!=5×4×3×2×1=120

$$C_5^2 = \frac{5!}{2!\cdot(5-2)!} = \frac{120}{2\cdot 6} = 10$$

CODE MINH HỌA

TINHTOHOP.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

long long giaithua(int n)
{
    long long T=1;
    for(int i=2;i<=n;i++)
        T *= i;
    return T;
}
long long Tohop(int n, int k)
{
    return giaithua(n)/(giaithua(k)*giaithua(n-k));
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    cin >> n >> k;
    cout << Tohop(n, k) << endl;

    return 0;
}
```

Công thức tối ưu không dùng giai thừa:

$$C_n^k = \frac{n\cdot(n-1)\cdot(n-2)\cdot\ldots\cdot(n-k+1)}{k!}$$

Hoặc tính trực tiếp như sau:

$$C_n^k = 1 \times \frac{n-k+1}{1} \times \frac{n-k+2}{2} \times \ldots \times \frac{n}{k}$$

Cách này **không cần tính từng giai thừa riêng lẻ**, mà tính dần kết quả theo từng bước.

Giải thích công thức

Phân tích tử số và mẫu số:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

Ta viết lại tử số: $n! = n\cdot(n-1)\cdot(n-2)\cdot\ldots\cdot(n-k+1)\cdot(n-k)!$

Nên: $C_n^k = \frac{n!}{k!(n-k)!} = \frac{n\cdot(n-1)\cdot\ldots\cdot(n-k+1)}{k!}$

Đưa về công thức nhân từng bước

$$C_n^k = 1\cdot\frac{n-k+1}{1}\cdot\frac{n-k+2}{2}\cdot\ldots\cdot\frac{n}{k}$$

mỗi bước lấy 1 cặp số ở tử và mẫu, chia và nhân dồn dần.

$$\frac{n \cdot (n-1) \cdot \ldots \cdot (n-k+1)}{k!} = \left(\frac{n-k+1}{1}\right) \cdot \left(\frac{n-k+2}{2}\right) \cdot \ldots \cdot \left(\frac{n}{k}\right)$$

CODE MINH HỌA

TINHTOHOP.cpp | [*] TINHTOHOP_v2.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

long long Tohop(int n, int k)
{
    if(k > n - k)
        k = n - k; // Tối ưu: C(n, k) = C(n, n - k)

    long long kq=1;
    for(int i=1;i<=k;i++)
    {
        kq *= (n - i + 1);
        kq /= i;
    }
    return kq;
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, k;
    cin >> n >> k;
    cout << Tohop(n, k) << endl;
    return 0;
}
```

2. VL18 - Tìm số đảo ngược

☺ VL18 - Tìm số đảo ngược

Viết chương trình tìm số đảo ngược của 1 số nguyên dương n nhập từ bàn phím.

Giới hạn

- $0 < n \leq 10^{1000}$

Input

Số nguyên dương n

Output

Số nguyên dương đảo ngược của số n

Sample

Input #1

```
1234
```

Output #1

```
4321
```

Input #2

```
1320
```

Output #2

```
231
```

Bài toán yêu cầu đảo ngược một số nguyên dương rất lớn (có thể dài đến 1000 chữ số), nên không thể dùng kiểu int hay long long trong C++. Thay vào đó, ta sẽ xử lý chuỗi (string).

CODE MINH HỌA – v1 – CÀI HÀM TÌM SỐ ĐẢO NGƯỢC (60% TEST)

```cpp
#include <bits/stdc++.h>
using namespace std;
long long sodn(long long n)
{
    long long dn=0;
    while(n!=0)
    {
        int dv=n%10;
        dn=dn*10+dv;
        n /=10;
    }
    return dn;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    long long n;
    cin>>n;
    cout<<sodn(n);
    return 0;
}
```

Bài nộp Tìm số đảo ngược

Xem code
Nộp lại

Kết quả

✔ ✔ ✔ ✘ ✘

- **Test case #1:** AC [0,002s,3.38 MB] (1/1)
- **Test case #2:** AC [0,002s,3.25 MB] (1/1)
- **Test case #3:** AC [0,002s,3.38 MB] (1/1)
- **Test case #4:** WA [0,002s,3.50 MB] (0/1)
- **Test case #5:** WA [0,002s,3.38 MB] (0/1)

Tài nguyên: 0,002s, 3.50 MB
Điểm cuối cùng: 3/5 (0,600/1,000 điểm)

CODE MINH HỌA – v2 – DÙNG STRING

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    string n;//n=1000
    cin>>n;
    reverse(n.begin(),n.end());// n=0001

    //Bỏ các số 0 ở đầu nếu có
    int i=0;
    while(n[i]=='0')
        i++;
    cout<<n.substr(i);// in ra dãy con của n bắt đầu từ vị trí i trở về sau.
    return 0;
}
```

Bài nộp Tìm số đảo ngược

Xem code
Nộp lại

Kết quả

✔ ✔ ✔ ✔ ✔

- **Test case #1:** AC [0,003s,3.50 MB] (1/1)
- **Test case #2:** AC [0,002s,3.38 MB] (1/1)
- **Test case #3:** AC [0,002s,3.38 MB] (1/1)
- **Test case #4:** AC [0,002s,3.38 MB] (1/1)
- **Test case #5:** AC [0,002s,3.38 MB] (1/1)

Tài nguyên: 0,003s, 3.50 MB
Điểm cuối cùng: 5/5 (1,000/1,000 điểm)